

University
Politehnica
of Bucharest





Computer
Science and
Engineering
Department

The NoSQL Movement
Key-Value Databases: Riak

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- The NoSQL Movement
- Key-Value Databases: Riak



Overview

- The NoSQL Movement
- Key-Value Databases: Riak



The NoSQL Movement

- The term NoSQL was first used in 1998 for relational database that omitted the use of SQL
- The term NoSQL was picked up again in 2009 by the advocates of non-relational databases
- The main reason for the NoSQL movement: to seek alternatives for solving problems for wich relational databases are a bad fit



The NoSQL Movement
Motives

1. Avoidance of unneeded complexity:
   - Relational databases provide a variety of features and strict data consistency
   - Relational databases use the ACID properties for transactions which are for some applications restrictive
   - The features, strict consistency and ACID properties of relational databases might be more than necessary for particular applications and use cases



The NoSQL Movement Motives

2. High throughput:

   - The features, strict consistency and ACID properties restrict the throughput
   - NoSQL use looser consistency and different less restrictive transaction properties to improve throughput
   - NoSQL DBMSs use different storage methods than RDBMSs to improve throughput
   - NoSQL DBMSs' processing capabilities are improved using distributed and parallel algorithms, i.e. MapReduce



The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:

   – Due to the huge volume of data, new technologies are needed to store and process this data

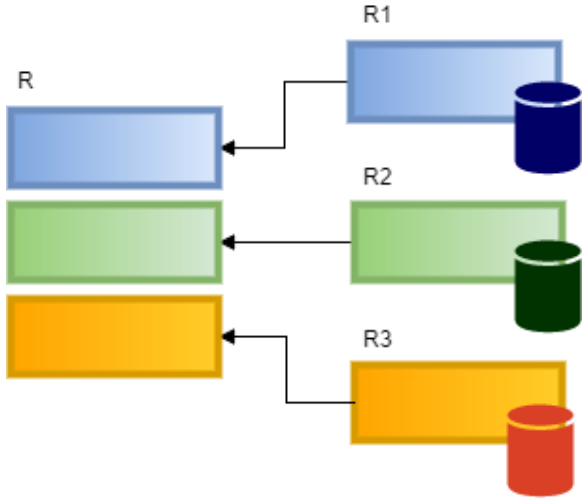
   – NoSQL DBMSs try to address the following current problems that DBMSs have:
     - Problems that appear when scaling out data due to the ACID properties
     - Performance of single servers
     - Processing of data on single servers
     - Rigid schema design
     - Costs of scaling, i.e. for Oracle, one needs at least Oracle Clusterware, Oracle Grid Infrastructure and Oracle Real Application Server on top of which the database is installed

The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Horizontal (Sharding)

Fragmentarea pe orizontală.

În fragmentarea pe orizontală (Figura 4.1a), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărțită relația R utilizează operația de reuniune pentru a reconstrui relația inițială, astfel $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$.

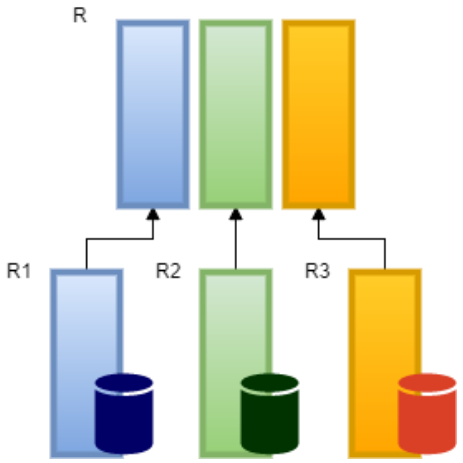


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Vertical (Sharding)

Fragmentarea pe verticală.

În fragmentarea pe verticală (Figura 4.1b), fragmentele R_i, $i = \overline{1,n}$, în care a fost împărțită relația R utilizează operația de JOIN pentru a reconstrui relația inițială, astfel $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$.

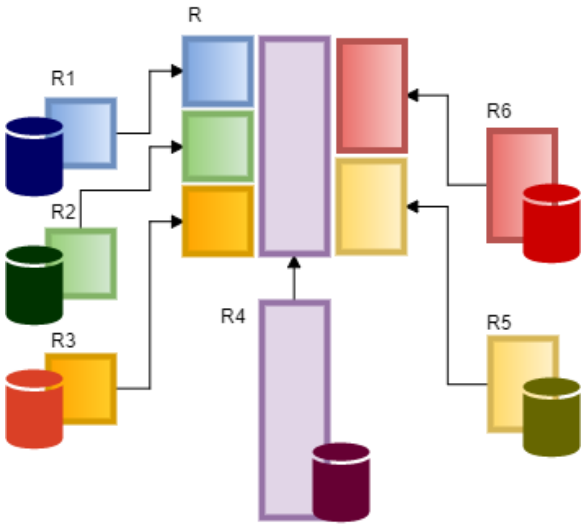


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Vertical-Horizontal (Sharding)

Fragmentarea mixtă verticală-orizontală.

În fragmentare mixtă verticală-orizontală (Figura 4.2a), relația R este mai întâi fragmentată pe verticală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$, și apoi fiecare relație R_i este la rândul ei fragmentată pe orizontală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstrui fragmentele R_i se va utiliza operația de reuniune astfel: $R_i = R_{i1} \cup R_{i2} \cup \ldots \cup R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de reuniune și apoi cea de JOIN: $R = (R_{11} \cup R_{12} \cup \ldots \cup R_{1n_1}) \bowtie (R_{21} \cup R_{22} \cup \ldots \cup R_{2n_2}) \bowtie \ldots \bowtie (R_{n1} \cup R_{n2} \cup \ldots \cup R_{nm_n})$.

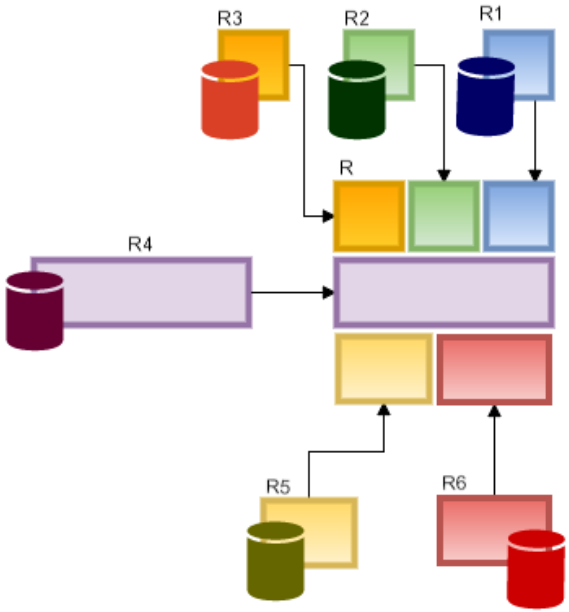


The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:
   - Fragmentation Mixt Horizontal-Vertical (Sharding)

Fragmentarea mixtă orizontală-verticală.

În fragmentare mixtă orizontală-verticală (Figura 4.2b), relația R este mai întâi fragmentată pe orizontală în relațiile R_i, $i = \overline{1,n}$, astfel încât $R = R_1 \cup R_2 \cup \ldots \cup R_n = \bigcup_{i=1}^{n} R_i$, și apoi fiecare relație R_i este la rândul ei fragmentată pe verticală în fragmentele R_{ij}, $j = \overline{1,m_i}$. Pentru a reconstruii fragmentele R_i se va utiliza operația de JOIN astfel: $R_i = R_{i1} \bowtie R_{i2} \bowtie \ldots \bowtie R_{im_i}$. Relația R este reconstruită utilizând mai întâi operația de JOIN și apoi cea de reuniunea: $R = (R_{11} \bowtie R_{12} \bowtie \ldots \bowtie R_{1n_1}) \cup (R_{21} \bowtie R_{22} \bowtie \ldots \bowtie R_{2n_2}) \cup \ldots \cup (R_{n1} \bowtie R_{n2} \bowtie \ldots \bowtie R_{nm_n})$.





The NoSQL Movement Motives

3. Horizontal scaling and running on commodity hardware:

   – NoSQL DBMSs are designed to:
   - Easily scale out (horizontal scale)
     – Machines can be added and remove without causing the same operational efforts to perform sharding in RDBMS cluster-solutions
   - Not rely on highly available hardware
   - Flexible schema-less design
   - Lower costs of scaling



The NoSQL Movement
Motives

4. Avoidance of Expensive object-relational mapping:

   – Most NoSQL DBMSs are designed to store data that are either simple or more like the ones of object-oriented programing languages compared to relational data structures

   – There is no need for expensive object-relational mapping

   – This is particularly important for applications with data structures of low complexity that can hardly benefit from the features of a relational database



The NoSQL Movement Motives

5. Decrease the Complexity and the Cost of Setting up Database Clusters:

   – NoSQL databases are designed in a way that PC clusters can be easily and cheaply expanded without the complexity and cost of sharding

   – Sharding involves cutting up databases into multiple tables to run on large clusters or grids



The NoSQL Movement
Motives

6. Compromising Reliability for Better Performance

- There are different scenarios for which applications can compromise reliability for better performance.
- E.g. HTTP session data which needs to be shared between various web servers but since the data is transient in nature (it goes away when the user logs off) there is no need to store it in persistent storage.



The NoSQL Movement Motives

7. The Current "One size fit's it all" Databases Thinking Was and Is Wrong

   – A growing number of application scenarios cannot be addressed with a traditional database approach, e.g. Facebook, Twitter, Instagram, etc.

   – The search for alternatives towards traditional RDBMSs can be explained by two major trends:
     - The continuous growth of data volumes (to be stored)
     - The growing need to process larger amounts of data in shorter time



The NoSQL Movement Motives

8. Movements in Programming Languages and Development Frameworks

- Multiple programming languages and development frameworks provide abstractions for database access trying to hide the use of SQL and relational databases

- The NoSQL databases react on this trend and try to provide data structures in their APIs that are closer to the ones of programming languages (e. g. key/value-structures, documents, graphs)

- Examples for this trend in the last couple of years include:
  - Object-relational mappers in the Java and .NET world like the Java Persistence API (JPA, part of the EJB 3 specification), implemented by e.g. Hibernate, or the LINQ Framework with the ADO.NET Entity Framework since .NET version 4
  - The popularity of Python and Ruby frameworks which hide the usage of relational databases



The NoSQL Movement Motives

9. Requirements of Cloud Computing
   - There are two major requirements of datastores in cloud computing environments:
     - High until almost ultimate scalability (especially in the horizontal direction)
     - Low administration overhead
   - The following classes of databases work well in the cloud:
     - Data warehousing specific databases for batch data processing and map/reduce operations
     - Simple, scalable and fast key/value-stores
     - Databases containing a richer feature set than key/value-stores fitting the gap with traditional RDBMSs while offering good performance and scalability properties, e.g. document-oriented databases



NoSQL Databases Categories

- By scalability
- By data and query model
- By persistence design



NoSQL Databases Categories Scalability

- Through replication, it is easy to scale:
  - Read operations
  - Load distribution
- The write operations are important for the scalability due to data distribution and partitioning
- For distributed systems that provide auto sharding there are 2 important features:
  - Support of multiple datacenters
  - Possibility to add machines live to an existing cluster, transparent to the applications and the users using the cluster



NoSQL Databases Categories
Data and query model

- Column-Oriented databases
- Key-Value databases
- Document-Oriented databases
- Graph databases



NoSQL Databases Categories Persistence design

1. In-Memory Databases

   - Are very fast
   - Tthe data size is inherently limited by the size of RAM
   - Durability may become a problem as the amount of data which can get lost between subsequent disk flushes is potentially large
     - e. g. due to server crashes or by losing power supply



NoSQL Databases Categories Persistence design

2. Memtables and SSTables work in the following way:

   - Write operations are buffered in memory (in a Memtable) after they have been written to an append-only commit log to ensure durability
   - After a certain amount of writes the Memtable gets flushed to disk as a whole to a SSTable (Idea taken from Google's Bigtable paper)
   - SSTable persistence strategy has performance characteristics comparable to those of in-memory-databases but avoids the durability difficulties of pure in-memory databases
   - Compared to disk-based strategies, disk seeks are reduced due to the append-only log and the flushing of whole Memtables to disk



NoSQL Databases Categories Persistence design

3. B-trees

- Have been used in databases since their beginning to provide a robust indexing-support
- Their performance characteristics on rotating disks are not very positive due to the amount of disk seeks needed for read and write operations
- The document-oriented database CouchDB uses B-trees internally but tries to avoid the overhead of disk seeks by only appending to B-trees (append-only B-trees):
  - Only one write operation at a time can happen
  - Concurrent reads to different sections of a B-tree are not allowed in this case

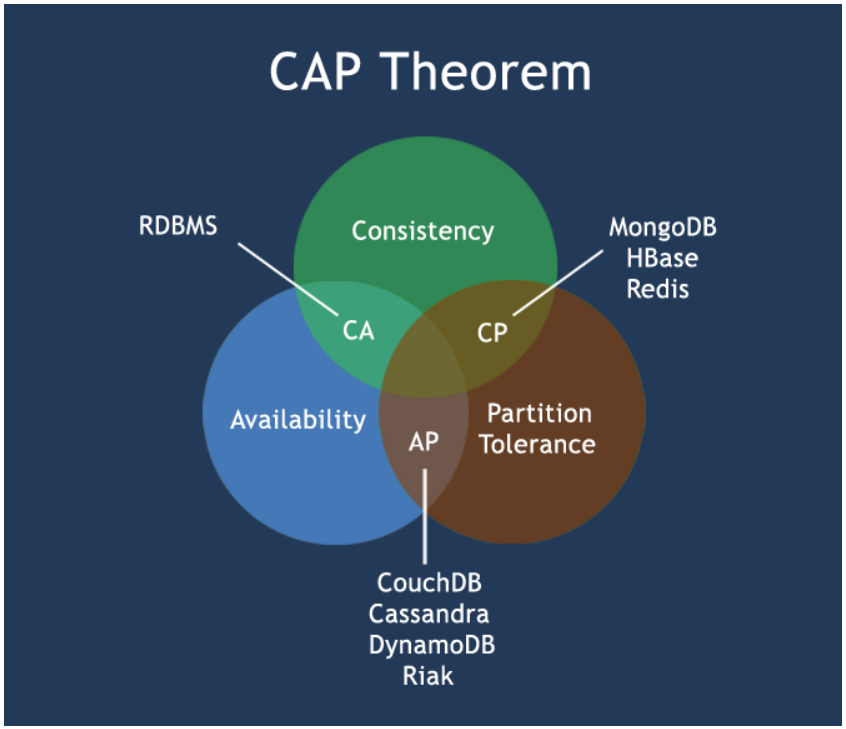


NoSQL Databases Categories

Type	Performance	Scalability	Flexibility	Complexity	Functionality
Key-Value	High	High	High	None	Variable
Column	High	High	Moderate	Low	Minimal
Document	High	Variable (high)	High	Low	Variable
Graph	Variable	Variable	High	High	Graph theory
Relational	Variable	Variable	Low	Moderate	Relational algebra

CAP Theorem

- Consistency
- Availability
- Partition Tolerance





CAP Theorem
Consistency

- Consistency meaning if and how a system is in a consistent state after the execution of an operation

- A distributed system is typically considered to be consistent if after an update operation of some writer all readers see his updates in some shared data source



CAP Theorem
Availability

- Availability and especially high availability meaning that a system is designed and implemented in a way that allows it to continue operation (i.e. allowing read and write operations) even if nodes in a cluster crash or some hardware or software parts are down due to upgrades.



CAP Theorem
Partition Tolerance

- Partition Tolerance is understood as the ability of the system to continue operation in the presence of network partitions

- These occur if two or more "islands" of network nodes arise which (temporarily or permanently) cannot connect to each other.

- Some people also understand partition tolerance as the ability of a system to cope with the dynamic addition and removal of nodes (e. g. for maintenance purposes; removed and again added nodes are considered an own network partition in this notion).



CAP Theorem

- Cap Theorem (Pick 2)

	Traits	Examples
Consistence + Availability	2-phase commit (2PC) Cache-validation protocols	RDBMS: Oracle, MySQL, MS SQL Server, PostgreSQL
Consistence + Partition Tolerance	Pessimistic locking Make minority partitions unavailable	MongoDB, HBase, Redis
Availability + Partition Tolerance	Conflict resolution Optimistic locking	CouchDB, Cassandra, Riak



CAP Theorem
Consistence + Availability

- 2-phase commit
  - A type of atomic commitment protocol
  - A distributed algorithm that coordinates all the processes that participate in a distributed atomic transaction on whether to commit or abort (roll back) the transaction

- Cache validation protocols:
  - Ensures cache coherence
  - Cache coherence is the uniformity of shared resource data that ends up stored in multiple local caches



CAP Theorem

- Minority partition (Consistence + Partition Tolerance)
  - A partition of nodes fewer or equal than half of the total number of nodes
  - Partions (islands) appear during communition failure between the nodes in a cluster

- Conflict resolution (Availability + Partition Tolerance)
  - Conflicts can occur in an environment that permits concurrent updates to the same data at multiple sites
  - Solutions: Multiversion concurrency control



CAP Theorem

- Pessimistic locking (Consistence + Partition Tolerance)
  - A resource
    - Is locked from the time it is first accessed in a transaction until the transaction is finished
    - Is inaccessible to other transactions during that time.
  - If most transactions simply look at the resource and never change it
    - An exclusive lock may be overkill as it may cause lock contention
    - Optimistic locking may be a better approach
  - Locks are applied in a fail-safe way
  - The lock exists until the transaction has either been committed or rolled back



CAP Theorem

- Optimistic locking (Availability + Partition Tolerance)
  - A resource is not actually locked when it is first accessed by a transaction
    - Other transactions can concurrently access the resource
    - Conflicting changes are possible
  - At commit time, when the resource is about to be updated in persistent storage the state of the resource
    - Is read from storage again
    - Is compared to the state that was saved when the resource was first accessed in the transaction
    - If the two states differ, a conflicting update was made, and the transaction will be rolled back



Transaction properties
ACID

- ACID (**A**tomicity, **C**onsistency, **I**solation, **D**urability)
  - **Atomicity**:
    - Requires that each transaction be "all or nothing"
    - If one part of the transaction fails, then the entire transaction fails, and the database state is left unchanged.
  - **Consistency** ensures that any transaction will bring the database from one valid state to another.
  - **Isolation** ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed sequentially, i.e., one after the other.
  - **Durability** ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.



Transaction properties BASE

- BASE (**B**asically **A**vailable, **S**oft-sate, **E**ventual consistency)
- **Basically Available:**
  - This constraint states that the system does guarantee the availability of the data as defined by the CAP Theorem.
  - There will be a response to any request
  - The response could still be 'failure' to obtain the requested data or the data may be in an inconsistent or changing state
- **Soft-state:**
  - The state of the system could change over time, so even during times without input there may be changes going on due to 'eventual consistency', thus the state of the system is always 'soft'
- **Eventual consistency:**
  - The system will eventually become consistent once it stops receiving input.
  - The data will propagate to everywhere it should sooner or later, but the system will continue to receive input and is not checking the consistency of every transaction before it moves onto the next one



ACID vs BASE

ACID	BASE
Strong consistency	Weak consistency
Isolation	Availability first
Focus on “commit”	Best effort
Real answers	Approximate answers
Availability	Aggressive Availability
Difficulty in modeling	Easier modeling



Overview

- The NoSQL Movement
- Key-Value Databases: Riak



Key-Value Databases

- Have a simple data model: map/dictionary
- Allows clients to request values by key
- Favor high availability over consistency
- Simple queries, they omit rich ad-hoc querying and analytics features – joins and aggregations are set aside
- The length of the keys are limited by a certain number of bytes while there is less limitation in values
- A great influence on the architecture is Amazon Dynamo Database



Riak

- Is a distributed key-value store
- Values can be anything from pain text, JSON, or XML to Images of video clips
- All the values are accessible through a simple HTTP interface
- Is fault tolerant – nodes can go up and down at any moment without any single point of failure
- A Riak cluster works continuously as nodes are added, removed or (ideally not) crush



Riak

- Riak trade-offs:
  - Lacks robust support for ad-hoc queries
  - Trouble linking values together (no foreign keys)



Riak

- Loves the web:
  - Querying is done via URLs, headers, and verbs
  - Riak returns assets and standard HTTP response codes
- Riak supplies an HTTP REST interface
- The interaction is done via the URL tool cURL
- cURL allows the interaction with the REST API without resorting to a particular driver or programming language



Riak

- Riak is a great choice for datacenters that must serve many requests with low latency
- It’s easy to manage, easy to set up, and can grow with your needs
- It is built in Erlang
- Can be installed on different distributions of Linux and Mac OS



Riak

- Riak's default mode of operation is to work as a cluster consisting of multiple nodes, i.e. multiple well-connected data hosts

- Each host in the cluster runs a single instance of Riak, referred to as a Riak node

- Each Riak node manages a set of virtual nodes (vnodes) that are responsible for storing a separate portion of the keys stored in the cluster



Riak

- Riak nodes (in contrast to some high-availability systems)
  - Are not clones of one another
  - Do not all participate in fulfilling every request
- At runtime or at request time, you can configure:
  - The number of nodes on which data is to be replicated
  - When replication occurs
  - Which merge strategy and failure model are to be followed.



Riak

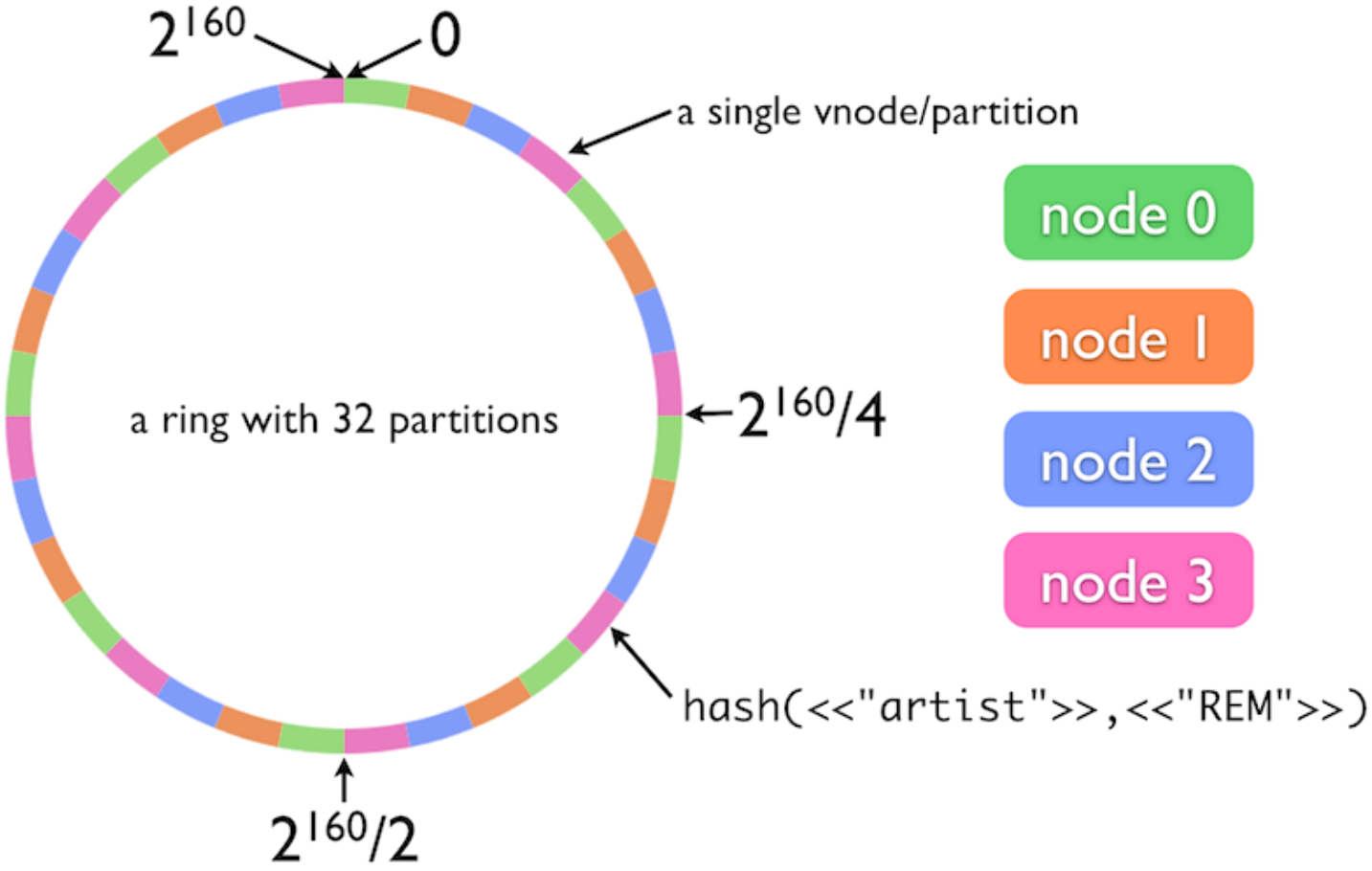
- Any client interface to Riak interacts with objects in terms of the bucket and key in which a value is stored (as well as the bucket type that is used to set the bucket's properties)

- Internally,
  - Riak computes a 160-bit binary hash of each bucket/key pair
  - Maps the hash value to a position on an ordered ring of all such values

- This ring is divided into partitions

- Each Riak **vnode** responsible for one of these partitions (meaning that each **vnode** claims that partition)

Riak


2^{160}
0
a single vnode/partition
node 0
node 1
node 2
node 3
a ring with 32 partitions
$2^{160}/4$
hash(<<"artist">>,<<"REM">>)
$2^{160}/2$



Riak

- The nodes of a Riak cluster each attempt to run a roughly equal number of **vnodes** at any given time
- In the general case, this means that each node in the cluster is responsible for
  - 1/(number of nodes) of the ring
  - (number of partitions)/(number of nodes) vnodes
- E.g. If two nodes define a 16-partition cluster then each node will run 8 vnodes
- Nodes attempt to claim their partitions at intervals around the ring such that
  - There is an even distribution amongst the member nodes
  - No node is responsible for more than one replica of a key



Riak
Intelligent Replication

- When an object is being stored in the cluster, any node may participate as the coordinating node for the request

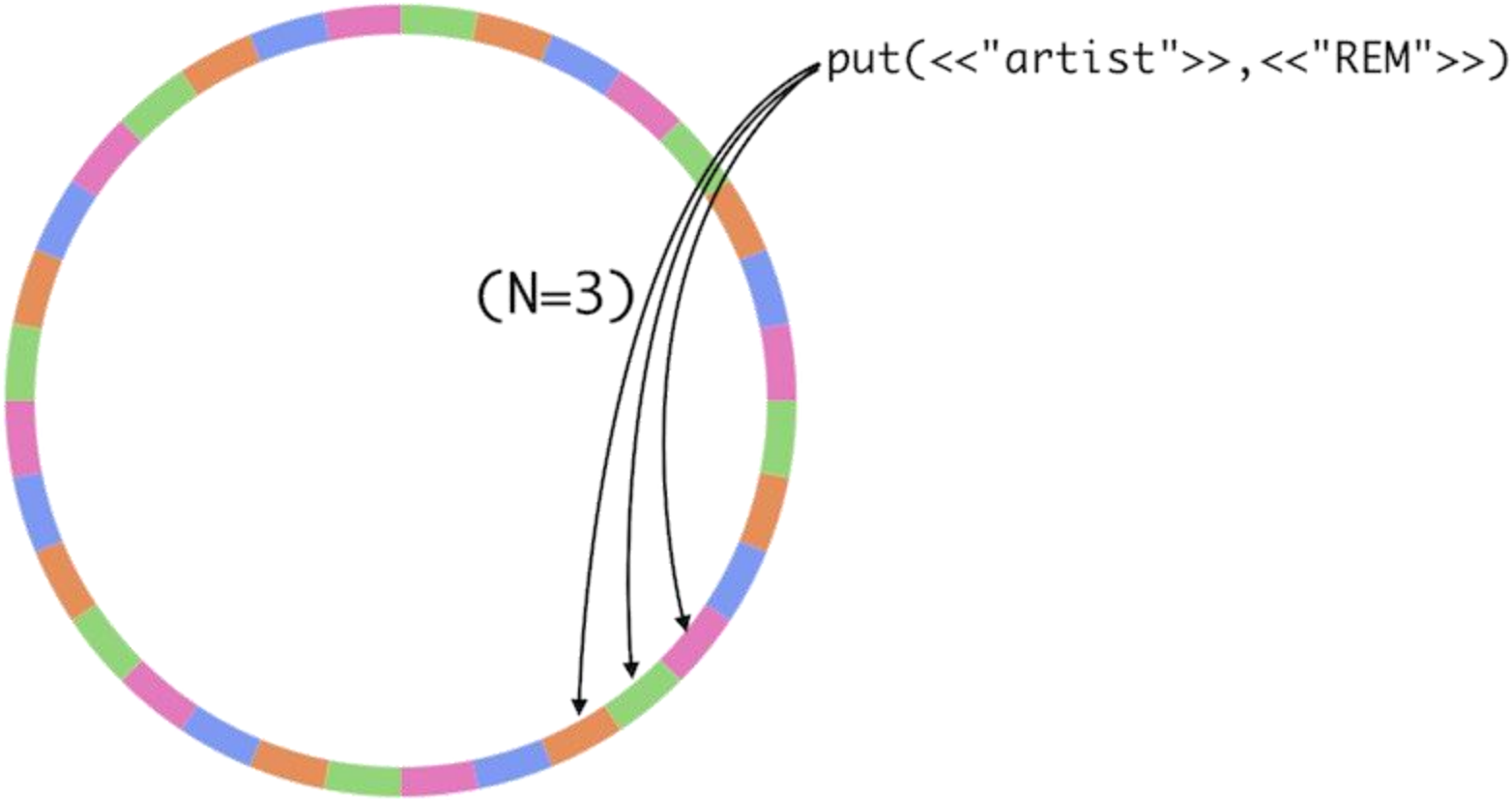
- The coordinating node
  - Consults the ring state to determine which vnode owns the partition in which the value's key belongs
  - Sends the write request to that vnode as well as to the vnodes responsible for the next N-1 partitions in the ring (where N is a configurable parameter that describes how many copies of the value to store)



Riak
Intelligent Replication

- The write request may also specify that
  - At least W (=< N) of those vnodes reply with success
  - DW (=< W) nodes reply with success only after durably storing the value
- A read, or GET, request operates similarly, sending requests
  - To the vnode that "claims" the partition in which the key resides
  - As well as to the next N-1 partitions
- The request also specifies R (=< N), the number of vnodes that must reply before a response is returned

Riak
Intelligent Replication





Riak
Intelligent Replication

- When N is set to 3, the value REM is stored in the key artist
- That key is assigned to 3 partitions out of 32 available partitions
- When a read request is made to Riak, the ring state will be used to determine which partitions are responsible
- From there, a variety of configurable parameters determine how Riak will behave in case the value is not immediately found



Intelligent Replication
Gossiping

- The ring state is shared around the cluster by means of a "gossip protocol"
- Whenever a node changes its claim on the ring, it announces, i.e. "gossips", this change to other nodes so that the other nodes can respond appropriately
- Nodes also periodically re-announce what they know about ring in case any nodes happened to miss previous updates



Installing Riak

- Installing Riak
  - Go to the official web site http://basho.com/products/ and download the package for your distribution.
  - You can also compile the source code but you need to install Erlang and additional packages.



Installing Riak

- Installing Riak on Ubuntu 18.04:
  - Download the latest version:

    wget --content-disposition https://packagecloud.io/basho/riak/packages/ubuntu/xenial/riak_2.2.3-1_amd64.deb/download.deb

  - Install the package

    sudo dpkg -i riak_2.2.3-1_amd64.deb

  - Start a node on localhost (wither with sudo privileges or as root)

    sudo riak start



Installing Riak on a cluster

- Download and install the package on all the nodes - do not start Riak
- On the node that will act as a master node modify the following lines in the riak.conf file (the default location is /etc/riak/riak.conf)
- Give a name to the Erlang node (nodename)

  nodename = riak@ip_host
- Set the IP Address and TCP port to bind Riak's HTTP interface

  listener.http.internal = ip_host:port
- Set the IP Address and TCP port to bind Riak's Protocol Buffers interface (Protocol Buffers is a method to serialize data)

  listener.protobuf.internal = ip_host:port
- Start the node

  $ sudo riak start



Installing Riak on a cluster

- Edit riak.conf and add IPs and Ports

$ sudo nano /etc/riak/riak.conf

```
nodename = riak@172.19.3.36
listener.protobuf.internal = 172.19.3.36:8087
listener.http.internal = 172.19.3.36:8098
```

$ sudo riak start



Installing Riak on a cluster

- In the case you see the following error because the riak node didn't start:

```
$ sudo riak start
riak failed to start within 15 seconds, see the output of 'riak console' for more
information. If you want to wait longer, set the environment variable
WAIT_FOR_ERLANG to the number of seconds to wait.
```

- Make sure that the configurations are correct

- Use the following command to force the reassignment of the IP

```
$ sudo riak-admin reip riak@127.0.0.1 riak@172.19.3.36
```

- Usually the error appears because the server was first started using the ‘localhost’ and some of the internal configuration files where set with the localhost IP (127.0.0.1)



Installing Riak on a cluster

- If the master node is now properly set up proceed to adding other nodes to the cluster by doing the same configurations as for the master

- The new nodes must join the cluster, on each node run the following command

  $ sudo riak-admin cluster join riak@ip_master

- In our example:

  $ sudo riak-admin cluster join riak@172.19.3.36

- Next, plan and commit the changes on each slave:

  $ sudo riak-admin cluster plan

  $ sudo riak-admin cluster commit

Installing Riak on a cluster

- The command line administrator tool is: riak-admin

- To check the cluster status

  $ sudo riak-admin cluster status

```
---- Cluster Status ----
Ring ready: true

+----------------------+------+-------+-----+-------+
|         node         |status| avail |ring |pending|
+----------------------+------+-------+-----+-------+
|     riak@172.19.3.31 |valid |   up  | 17.2|   --  |
|     riak@172.19.3.32 |valid |   up  | 17.2|   --  |
|     riak@172.19.3.33 |valid |   up  | 17.2|   --  |
|     riak@172.19.3.34 |valid |   up  | 15.6|   --  |
|     riak@172.19.3.35 |valid |   up  | 17.2|   --  |
| (C) riak@172.19.3.36 |valid |   up  | 15.6|   --  |
+----------------------+------+-------+-----+-------+

Key: (C) = Claimant; availability marked with '!' is unexpected
```

Installing Riak on a cluster

- To see partitions assigned for each node

  $ sudo riak-admin cluster partitions

```
Partitions owned by 'riak@172.19.3.36':
+---------+-----------------------------------------------------+--+
|  type   |                        index                        |id|
+---------+-----------------------------------------------------+--+
| primary |20552366774965822287239317960072729963911551385 6 |9 |
| primary |31970348316613501335705605715668691054973524377 6 |14|
| primary |43388329858261180384171893471264652146035497369 6 |19|
| primary |66224292941556538481104468982456574328159443353 6 |29|
| primary |77642274483204217529570756738052535419221416345 6 |34|
| primary |89060256024851896578037044493648496510283389337 6 |39|
| primary |10047823756649957562650333224924445760134536232 96|44|
| primary |11189621910814725467496962000484041869240733532 16|49|
| primary |12331420064979493372343590776043637978346930831 36|54|
| primary |13473218219144261277190219551603234087453128130 56|59|
|secondary|                        --                           |--|
| stopped |                        --                           |--|
+---------+-----------------------------------------------------+--+
```



Riak Statistics

- To see the statistics of the server

$ sudo riak-admin status

- cURL can be used to get the statistics:

$ curl -GET http://172.19.3.36:8098/stats

- To check Riak configuration

$ sudo riak chkconfig

- Ping the cluster to see if it's up and running:

$ sudo riak ping

$ curl -GET http://172.19.3.36:8098/ping



Riak control interface

- Riak provides a HTTP control interface
- By default is not enabled
- To enable it without authentication, you need to edit the riak.conf file and change the value of parameter riak_control from off to on:

  riak_control = on



Riak control interface

- To add authentication to the control interface
  - SSL must be enabled and set up properly on each Riak node
  - Need to enable authentication by changing the value of parameter riak_control.auth.mode from of to userlist

    riak_control.auth.mode = userlist

  - Add a list of users where the value is the password

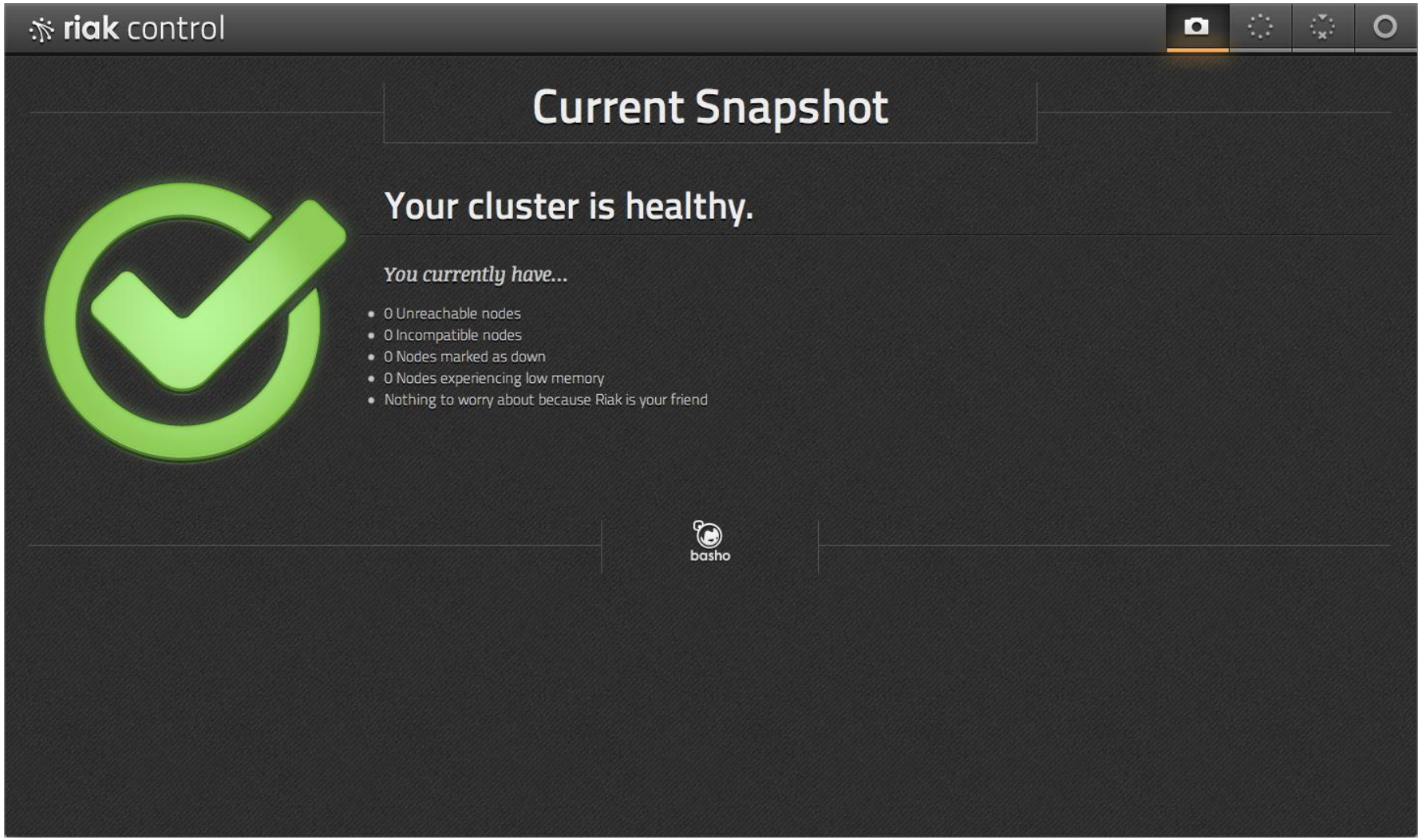
    riak_control.auth.user.bob.password = bob_is_the_coolest



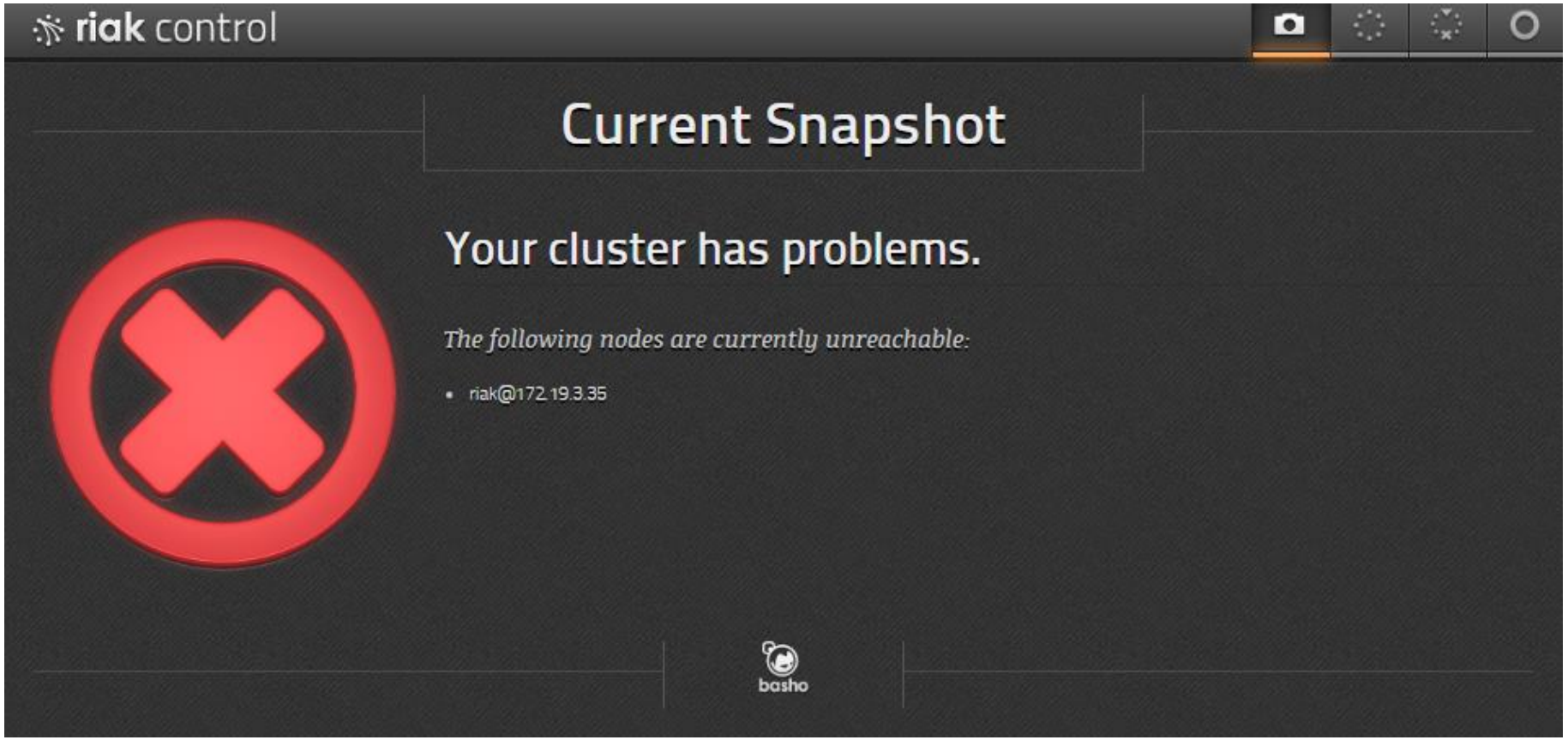
Riak control interface Snapshot view

- In this interface, the health of your cluster is made immediately obvious
- If something isn't quite right (or has the potential to cause problems soon), the green check mark will turn into a red X.
- The red X is accompanied by a list of reasons for concern.
- Each item in the list links to a page where you can get more information about the issue

Riak control interface
Snapshot view



Riak control interface
Snapshot view

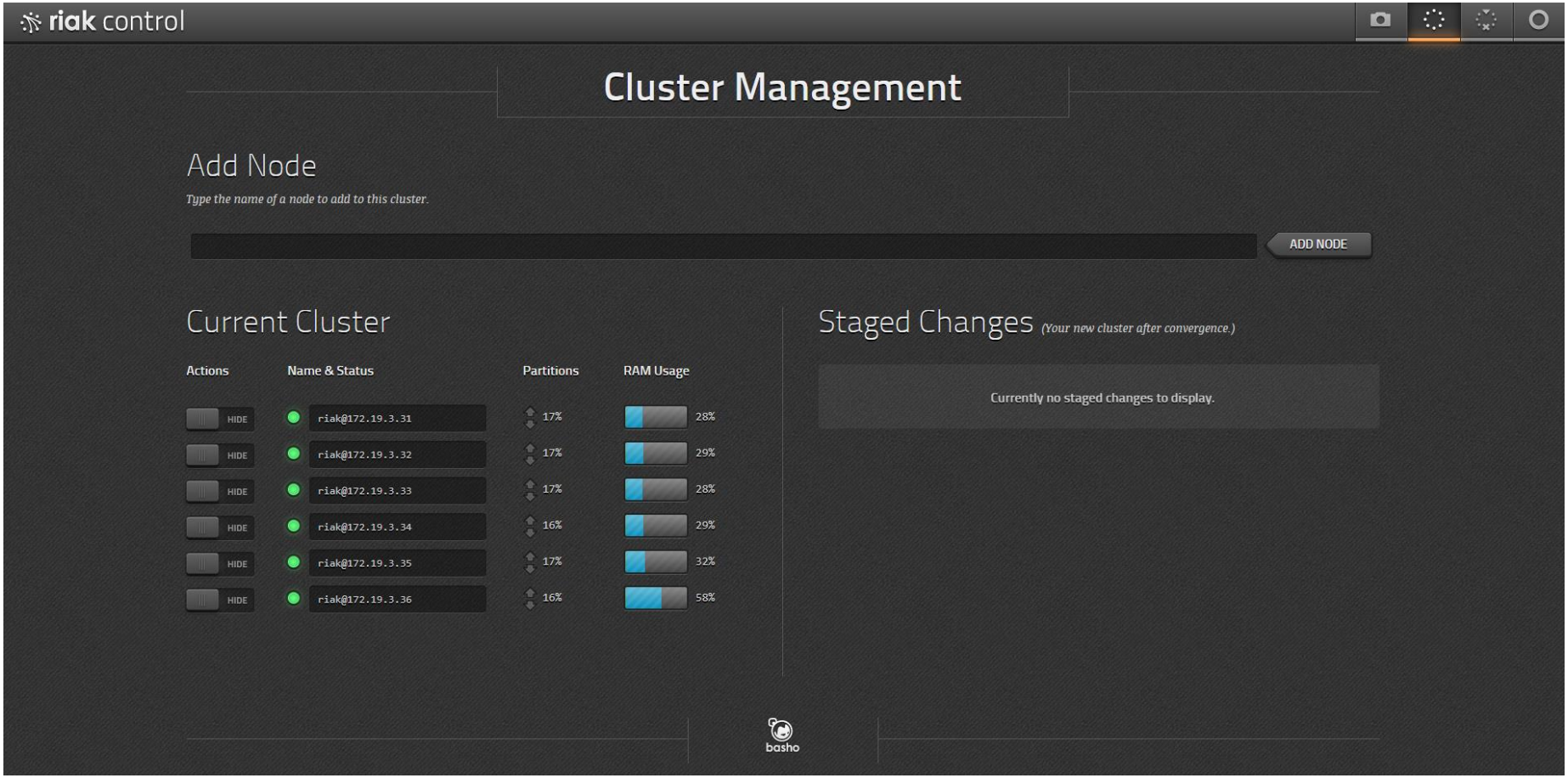




Riak control interface
Cluster Management view

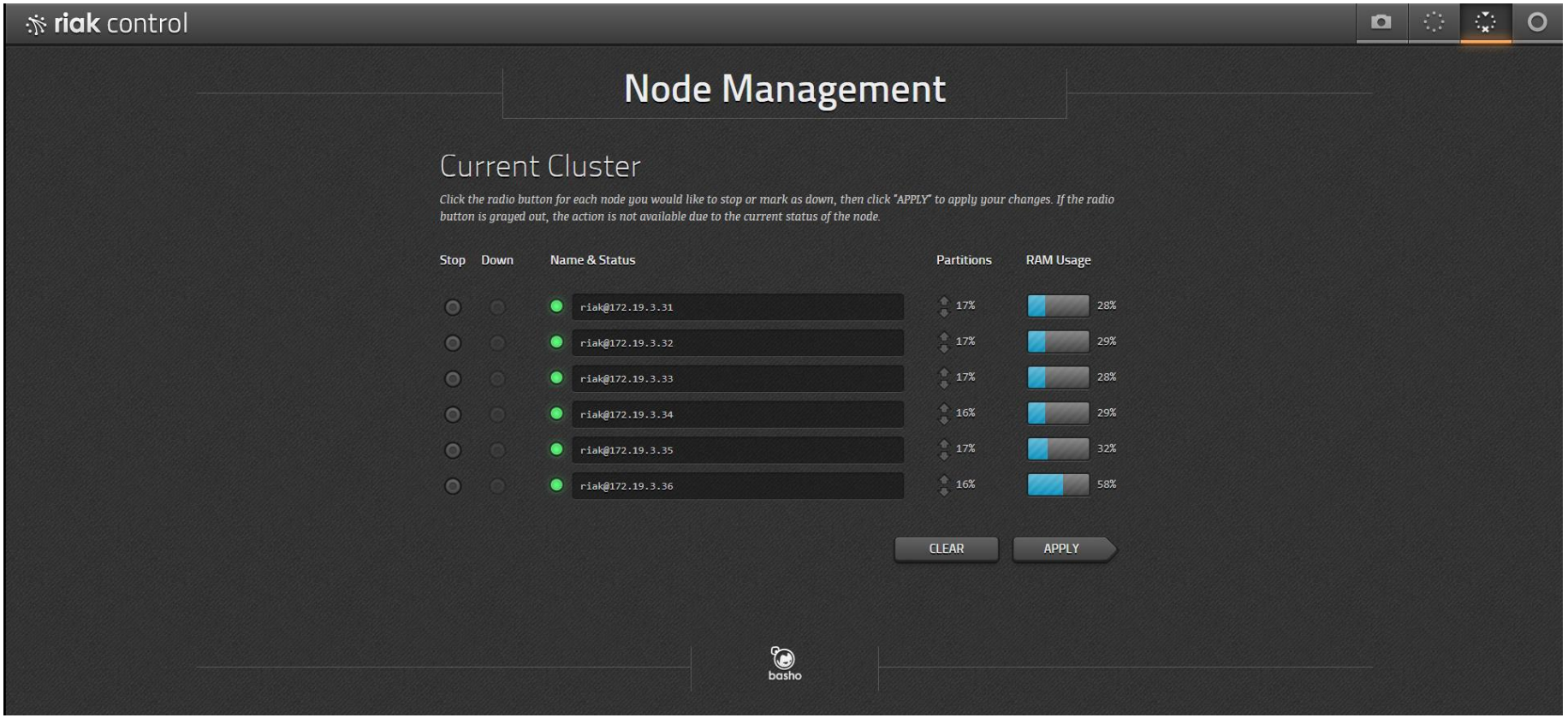
- See all the nodes in your cluster
  - Node status
  - Percentage of the ring owned by that node
  - Memory consumption
- Stage and commit changes to the cluster
  - Adding nodes
  - Removing nodes
  - Marking nodes as down

Riak control interface
Cluster Management view



Riak control interface
Node Management view

- The node management view allows you to operate against the individual nodes in the cluster

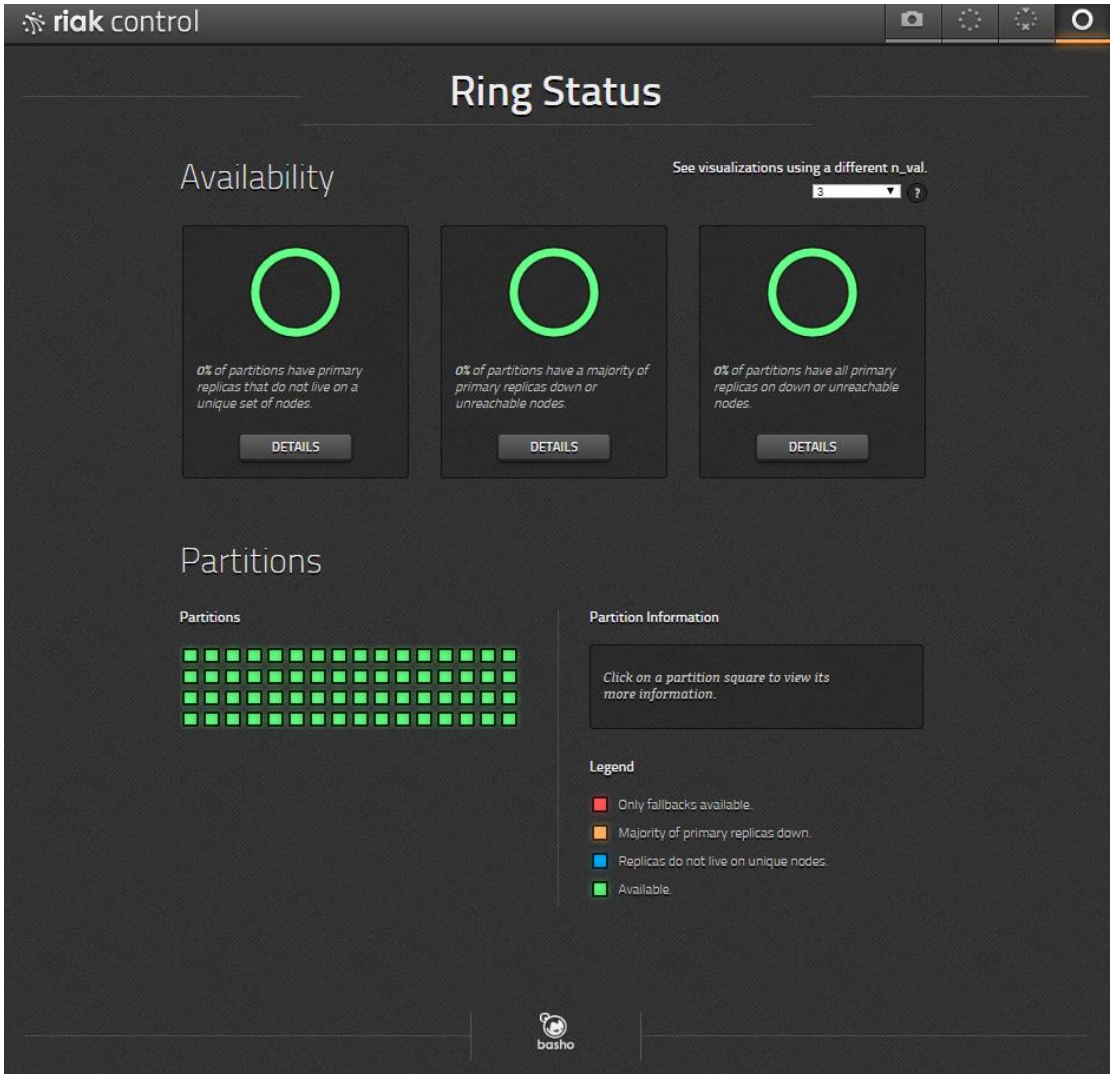




Riak control interface
Ring Status view

- This is where you can see the health of each vnode
- Most of the time, your ring will be too large to effectively manage from the ring view
- With filters you can easily identify
  - Partition ownership
  - Unreachable primaries
  - In-progress handoffs.

Riak control interface
Ring Status view





Riak and cURL

- REST stands for REpresentational State Transfer
- REST is a guideline for mapping resources to URLs and interacting with them using CRUD verbs:
  - POST (**C**reate)
  - GET (**R**ead)
  - PUT (**U**pdate)
  - DELETE (**D**elete).
- cURL is used to directly connect to Riak server's HTTP REST interface without the need for an interactive console



Riak and cURL

- If we want only the header response use –I

$ curl -I http://172.19.3.36:8098/riak/animals

```
HTTP/1.1 200 OK
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Date: Sat, 18 Nov 2017 17:51:41 GMT
Content-Type: application/json
Content-Length: 464
```



Riak and cURL
PUT

- Adding a new key-value element
  - Use curl with -X PUT or -XPUT
  - The -H attribute sets the following text as HTTP header information. In the example, we set the MIME content type to HTML.
  - Everything passed to -d (also known as the body data) is what Riak will add as a new value.
  - The -v attribute stands for verbose
  - The bucket is favs (equivalent to a schema)
  - The key is db

```
$ curl -v -X PUT http://172.19.3.36:8098/riak/favs/db \
-H "Content-Type: text/html" \
-d "<html><body><h1>My new favorite DB is RIAK</h1></body></html>“
```

  - Check http://172.19.3.36:8098/riak/favs/db



Riak and cURL
POST

- The attribute POST is used if we want the system to generate a new key by itself

```
curl -v -X POST http://172.19.3.36:8098/riak/favs \
-H "Content-Type: text/html" \
-d "<html><body><h1>Auto generated key</h1></body></html>"
```



Riak and cURL
POST

```
ciprian@hadoop-master:~$ curl -v -X POST http://172.19.3.36:8098/riak/favs \
> -H "Content-Type: text/html" \
> -d "<html><body><h1>Auto generated key</h1></body></html>"
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 172.19.3.36...
* Connected to 172.19.3.36 (172.19.3.36) port 8098 (#0)
> POST /riak/favs HTTP/1.1
> Host: 172.19.3.36:8098
> User-Agent: curl/7.47.0
> Accept: */*
> Content-Type: text/html
> Content-Length: 53
>
* upload completely sent off: 53 out of 53 bytes
< HTTP/1.1 201 Created
< Vary: Accept-Encoding
< Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
< Location: /riak/favs/K42Xu7nxGdy7S7TlmVj5dY9P80
< Date: Sat, 18 Nov 2017 18:11:00 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.19.3.36 left intact
```

New Key



Riak and cURL
DELETE

- The DELETE attribute is used to delete a record

$ curl -v -X DELETE http://172.19.3.36:8098/riak/favs/K42Xu7nxGdy7S7TlmVj5dY9P80

```
*   Trying 172.19.3.36...
* Connected to 172.19.3.36 (172.19.3.36) port 8098 (#0)
> DELETE /riak/favs/K42Xu7nxGdy7S7TlmVj5dY9P80 HTTP/1.1
> Host: 172.19.3.36:8098
> User-Agent: curl/7.47.0
> Accept: */*
>
< HTTP/1.1 204 No Content
< Vary: Accept-Encoding
< Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
< Date: Sat, 18 Nov 2017 18:17:33 GMT
< Content-Type: text/html
< Content-Length: 0
<
* Connection #0 to host 172.19.3.36 left intact
```



Riak and cURL
GET

- The GET attribute is used to get the information for a record

$ curl -v -X GET http://172.19.3.36:8098/riak/favs/db

```
Note: Unnecessary use of -X or --request, GET is already inferred.
*   Trying 172.19.3.36...
* Connected to 172.19.3.36 (172.19.3.36) port 8098 (#0)
> GET /riak/favs/db HTTP/1.1
> Host: 172.19.3.36:8098
> User-Agent: curl/7.47.0
> Accept: */*
>
< HTTP/1.1 200 OK
< X-Riak-Vclock: a85hYGBgzGDKBVI8Z49e0F7+RUaPgcFPOYMpkTGPlUHuT8tlviwA
< Vary: Accept-Encoding
< Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
< Link: </riak/favs>; rel="up"
< Last-Modified: Sat, 18 Nov 2017 18:46:54 GMT
< ETag: "5rpxt7wosYhYGS4VJt4mCx"
< Date: Sat, 18 Nov 2017 18:46:57 GMT
< Content-Type: text/html
< Content-Length: 61
<
* Connection #0 to host 172.19.3.36 left intact
<html><body><h1>My new favorite DB is RIAK</h1></body></html>
```



Riak Example

- Example: Animal shelter
- Let’s create a shelter for animals and add 2 dogs

```
curl -v -X PUT http://172.19.3.36:8098/riak/animals/ace \
-H "Content-Type: application/json" \
-d '{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}'

curl -v -X PUT http://172.19.3.36:8098/riak/animals/polly?returnbody=true \
-H "Content-Type: application/json" \
-d '{"nickname" : "Sweet Polly Purebred", "breed" : "Purebred"}'
```



Riak links

- Links are metadata that associate one key to other keys

- The basic structure is this:

Link: </riak/bucket/key>; riaktag=\"whatever\“

- The key to where this value links is in pointy brackets (<…>), followed by a semicolon and then a tag describing how the link relates to this value (it can be whatever string we like).

- There can be many metadata links as necessary separated by commas

- Links are one-directional



Riak Example

- Let’s add 2 rooms for the dogs with links

```
curl -X PUT http://172.19.3.36:8098/riak/cages/1 \
-H "Content-Type: application/json" \
-H "Link: </riak/animals/polly>; riaktag=\"contains\"" \
-d '{"room" : 101}'

curl -X PUT http://172.19.3.36:8098/riak/cages/2 \
-H "Content-Type: application/json" \
-H "Link:</riak/animals/ace>;riaktag=\"contains\", </riak/cages/1>;riaktag=\"next_to\"" \
-d '{"room" : 102}'
```



Riak Example

- The links are seen only on the room records, not the animal records

curl -i http://172.19.3.36:8098/riak/animals/polly

curl -i http://172.19.3.36:8098/riak/animals/ace

curl -i http://172.19.3.36:8098/riak/cages/1

curl -i http://172.19.3.36:8098/riak/cages/2



Riak Example

```
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/animals/ace
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgymDKBVI8wQtdOxbpPrRlYFAXymBKZMpjZXgiOXaZDyp9aoWIObKdGjxAaUGo9EtVoHQWAA==
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:29 GMT
ETag: "4J49QrCprFeCtfU2DlNwZb"
Date: Sun, 19 Nov 2017 16:05:02 GMT
Content-Type: application/json
Content-Length: 60

{"nickname" : "The Wonder Dog", "breed" : "German Shepherd"}
ciprian@hadoop-master:~$ curl -i -w '\n' http://172.19.3.36:8098/riak/cages/2
HTTP/1.1 200 OK
X-Riak-Vclock: a85hYGBgzGDKBVI8Z49e0C65smo/A4OfcgZTInMeK8NX1bbLfFkA
Vary: Accept-Encoding
Server: MochiWeb/1.1 WebMachine/1.10.9 (cafe not found)
Link: </riak/animals/ace>; riaktag="contains", </riak/cages/1>; riaktag="next_to", </riak/cages>; rel="up"
Last-Modified: Sun, 19 Nov 2017 15:57:41 GMT
ETag: "4VPZpyDp4OFOUwpULVTW8A"
Date: Sun, 19 Nov 2017 16:05:08 GMT
Content-Type: application/json
Content-Length: 14

{"room" : 102}
```



Riak Link Walking

- Getting the linked data is achieved by appending a link spec to the URL that is structured like this: /_,_,_
- The underscores (_) in the URL represent wildcards to each of the link criteria:
  - Bucket
  - Tag (name of link)
  - Keep
- Keep (1 or 0) is used to keep the link when traversing multiple links
- It returns a multipart/mixed dump of headers plus bodies of all linked keys/values.
- Replace the underscores in the link spec to filter only values we want



Riak Example

curl http://172.19.3.36:8098/riak/cages/1/_,_,_

curl http://172.19.3.36:8098/riak/cages/1/_,contains,_

curl http://172.19.3.36:8098/riak/cages/2/_,contains,_

curl http://172.19.3.36:8098/riak/cages/2/_,next_to,_

curl http://172.19.3.36:8098/riak/cages/2/cages,next_to,1/animals,_,_

curl http://172.19.3.36:8098/riak/cages/2/cages,next_to,0/animals,_,_



Riak and MapReduce

- Riak support MapReduce for aggregation
- The Map and Reduce functions are written in JavaScript
- To run a map reduce function we need to use cURL on the /mapred link:

```
curl -X POST -H "content-type:application/json" http://172.19.3.36:8098/mapred --
data @-
{
   "inputs": “bucket",
   "query":[ { "map": { "language":"javascript",
               "source": "function(v) {…}“ } },
            { "reduce": { "language":"javascript",
               "source": "function(v) { …}“ } } ]
}
```



Riak Stored Functions

- We can store MapReduce functions

```
curl -X PUT -H "content-type:application/json" \
http://172.19.3.36:8098/riak/my_functions/map_capacity --data \
"function(v) { var parsed_data = JSON.parse(v.values[0].data); var data = {};
data[parsed_data.style] = parsed_data.capacity; return [data];}"
```



Riak Stored Functions

```
curl -X POST -H "content-type:application/json" http://172.19.3.36:8098/mapred --
data @-
{ "inputs": "rooms",
   "query":[
      { "map": {
         "language": "javascript",
         "bucket": "my_functions",
         "key": "map_capacity" }},
      { "reduce": {
         "language":"javascript",
         "bucket": "my_functions",
         "key": "reduce_capacity" }}
   ]
}
```



Riak
Key Filtering with MapReduce

- Riak supports filtering by key before using MapReduce
- To use filters, we need to add them in the input label:

```
"inputs":{
      "bucket":"bucket",
      "key_filters":[filters]
   },
```



Riak
Key Filtering with MapReduce

```
curl -X POST -H "content-type:application/json"
http://172.19.3.36:8098/mapred --data @-
{ "inputs": { "bucket": "rooms", "key_filters":[["string_to_int"], ["less_than",
1000]] },
 "query":[
  { "map": {
     "language": "javascript",
     "bucket": "my_functions",
     "key": "map_capacity" } },
  { "reduce": {
     "language":"javascript",
     "bucket": "my_functions",
     "key": "reduce_capacity" }}  ] }
```



Riak
MapReduce and Link Walking

- We can use link walking with MapReduce
- To do this we need to add a new tag called link in query label of the JSON

```
"query":[
    {
      "link":
        {
          "bucket":"animals",
          "keep":false
        }
    },
```



Riak
MapReduce and Link Walking

```
curl -X POST -H "content-type:application/json" \
http://172.19.3.36:8098/mapred --data @-
{ "inputs":{ "bucket":"cages", "key_filters":[["eq", "2"]]},
  "query": [
      { "link": {
            "bucket":"animals",
            "keep":false } },
      { "map": {
            "language":"javascript",
            "source":
            "function(v) { return [v]; }" } }   ] }
```